This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

From Uniform to Calibration: Priority-Conditioned Differential Privacy for Federated Vehicular Edge Client Selection

Yishan Chen, Decan Cheng, Mengfan Teng, Runshan Xie, Junxiao Han, Guanjie Cheng*

***Abstract*—Vehicular edge computing reduces transmission latency by moving computing resources closer to vehicles. Federated learning further limits raw-data transmission by enabling local model training and uploading only model updates. However, uploaded updates remain vulnerable to model inversion or membership inference attacks by honest-but-curious servers or external adversaries. Moreover, conventional differential privacy (DP) assigns a uniform privacy budget across tasks, failing to accommodate the diverse priorities and privacy requirements of vehicular applications and potentially degrading model utility for less-sensitive tasks. To address these issues, we propose a client selection and resource allocation strategy integrating federated learning with Vehicular Edge Computing, aiming to reduce system processing latency and energy consumption while preserving the user privacy. First, we formulate the client selection and resource allocation decision problem as a Markov decision process. The formulation considers task completion, latency, energy consumption, and model accuracy, while privacy protection is enforced by priority-conditioned DP. Task priority is used to calibrate the privacy budget of uploaded updates, and Laplace perturbation is applied after update clipping. FL_MADDPG is then used to learn adaptive client selection and resource allocation policies in dynamic vehicular environments. Experiments on multiple datasets show that the proposed framework reduces latency and energy consumption compared with representative DRL baselines while maintaining model utility under differentiated privacy protection.**

***Index Terms*—Privacy Preserving, client selection, edge computing, federated learning, personalized differential privacy.**

I. Introduction

With the rapid advancement of 5G and 6G, the Internet of Vehicles (IoV) has become a cornerstone for constructing future intelligent transportation systems [1], [2], [3], [4]. By enabling autonomous driving, real-time traffic analysis, high-definition mapping, and immersive in-vehicle entertainment, IoV dramatically enhances both traffic efficiency and driving safety. However, the real-time processing and analysis of massive vehicular data are constrained by the limited computational and storage resources of onboard units, as well as stringent latency and energy requirements [5], [6]. Centralized cloud computing can provide abundant computing resources, but the long physical distance between vehicles and cloud servers may introduce excessive transmission latency and network congestion for latency-sensitive services [7]. Therefore, Vehicular Edge Computing (VEC), also known as Mobile Edge Computing (MEC), emerged to bring computing and storage resources closer to vehicles. By deploying these resources closer to users at the network edge, VEC can significantly reduce task processing latency and backbone network bandwidth consumption, better meeting the strict requirements of IoV applications [8].

Nevertheless, edge resources are still limited and cannot support the simultaneous participation of all vehicles in federated training. Therefore, client selection is essential for federated training in VEC environments, where the system needs to select vehicles with high-quality model updates under constrained computing and communication resources. Such a decision also affects system efficiency because vehicular clients are heterogeneous in data distribution, computing capability, channel condition, and task requirements, and different selections may result in different training latency and energy consumption. Accordingly, designing an effective client selection strategy still faces two key challenges. Firstly, the high mobility of vehicles causes fluctuations in link quality, resulting in intermittent client availability and time-varying effective compute capacity under communication constraints, yielding a high-dimensional dynamic optimization problem for client selection decisions. Traditional optimization methods, such as greedy or integer programming algorithms, often struggle to adapt to such intricate dynamic environments. While some works have introduced Deep Reinforcement Learning (DRL) to address dynamic environments, challenges persist in achieving fine-grained resource allocation and multi-objective collaborative optimization [9], [10], [11].

Secondly, there remains an unavoidable privacy leakage risk. In Federated Learning (FL), selected vehicles upload model updates for aggregation, from which adversaries or honest-but-curious servers may infer sensitive driving information through gradient, model inversion, and membership inference attacks. The risk is closely related to the selection policy, because vehicles that are frequently selected or assigned larger aggregation weights expose more update information during training. Differential Privacy (DP) is commonly adopted to mitigate such risks by injecting noise into the shared updates. However, many DP mechanisms still apply a uniform privacy

Yishan Chen, Decan Cheng, Mengfan Teng, and Runshan Xie are with Jiangxi Provincial Key Laboratory of Multidimensional Intelligent Perception and Control, Jiangxi University of Science and Technology, Ganzhou 341000, China (email: chenys@jxust.edu.cn; chengdc@mail.jxust.edu.cn; tengmengfan@whu.edu.cn; runshan_xie@foxmail.com).

Junxiao Han is with the School of Computer and Computing Science, Hangzhou City University, Hangzhou 310015, China (email: hanjx@hzcu.edu.cn).

Guanjie Cheng is with the School of Software Technology, Zhejiang University, Hangzhou 310027, China (email: guanjiech@126.com).

*Corresponding author.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

budget to all uploaded model updates, which is difficult to match the diverse task priorities and privacy requirements. For instance, driving-safety and navigation tasks usually involve location, trajectory, and driving behavior information, and thus require stronger privacy protection, whereas entertainment tasks can usually tolerate weaker protection to preserve model utility. Therefore, the privacy budget of each uploaded model update should be calibrated according to task priority instead of being fixed uniformly. Otherwise, tasks with low privacy sensitivity may receive excessive noise, unnecessarily sacrificing model accuracy and system utility, while highly sensitive tasks may be insufficiently protected, leaving potential privacy risks unresolved [12].

To systematically address the dual challenges of dynamic decision optimization and fine-grained privacy protection, this paper proposes a novel client selection and resource allocation strategy for IoV that integrates Federated Learning and Edge Computing. The objective is to jointly optimize system latency and energy consumption while preserving the user data privacy and security. Our core contributions are as follows:

- We model the complex client selection decision-making and resource allocation problem as a Markov Decision Process (MDP). By designing a utility function that integrates task completion, latency, energy consumption, and model accuracy, the proposed formulation enables the system to optimize training participation and resource allocation under dynamic vehicular conditions.
- We propose a privacy calibration mechanism based on task priority and personalized differential privacy. By mapping task priority to privacy budgets, the proposed mechanism calibrates the privacy budgets of uploaded model updates according to their associated task priorities, thereby providing stronger protection for updates related to tasks with higher privacy sensitivity while avoiding unnecessary perturbation for less sensitive ones.
- We design an FL_MADDPG algorithm to solve the formulated dynamic decision problem. The feasibility and effectiveness of the proposed framework are verified through extensive experiments under different resource and privacy settings.

The remainder of the paper is organized as follows. Section II reviews related work. Section III constructs the system model and formalizes the problem definition. Section IV elaborates on our proposed algorithm. Section V presents the simulation results and analysis. Section VI concludes the paper and outlines future work.

II. Related Work

In this section, we review the existing literature, focusing specifically on client selection and privacy preservation in the FL process.

A. Client Selection in FL

Recent advances increasingly leverage environment-adaptive and DRL approaches to jointly optimize client selection and resource allocation in federated learning, addressing the limitations of static, rule-based scheduling under device and channel heterogeneity [13], [14], [15], [16]. Zheng et al. [13] propose FedAEB, the first framework to jointly encode device computational capability and wireless channel state into the DRL state space, enabling synchronous optimization of client scheduling and spectrum allocation and achieving up to 40% acceleration in convergence. Similarly, [14], [15] further extend DRL-based scheduling by embedding bandwidth and interference constraints into the action and reward spaces, achieving robust model performance even under stringent resource limitations and high uplink interference. Another line of research considers the impact of communication architectures and aggregation protocols on client selection, particularly in large-scale or hierarchical deployments [17], [18], [19]. Xu et al. [17] present CSTAR-FL, which embeds stochastic client sampling into a tree-based All-Reduce protocol, significantly reducing synchronization latency while maintaining convergence guarantees. Building on this, Shi et al. [18] develop a dynamic client scheduler for over-the-air aggregation that selects clients in real time according to channel gain and data freshness, thus improving wireless channel utilization. Furthermore, Chen et al. [19] expand the client selection scope to include edge server offloading and cross-layer resource coordination in Split FL, achieving a 30% reduction in end-to-end training latency.

Fairness-aware and generalization-oriented strategies form a parallel research thread, aiming to mitigate model bias caused by non-IID data distributions and unstable client participation [20], [21], [22], [23]. Tang et al. [20] introduce a class-balanced selection criterion that incorporates local data distribution statistics, ensuring minority-class representation and improving accuracy on long-tailed datasets. Other works, such as Tan et al. [21], employ multi-round auction mechanisms with incentive compatibility and long-term reputation scoring to suppress strategic dropout and stabilize participation rates. To further address fairness, [22], [23] propose group fairness constraints and bilevel optimization with sample-level screening, which help to balance accuracy across device groups and enhance convergence stability under significant data heterogeneity. Despite significant progress, many advanced selection strategies involve considerable computational or deployment overhead, limiting their practicality in highly resource-constrained or dynamic scenarios [24], [25], [26]. To address this, Pan et al. [24] adopt contextual bandit models based on device telemetry, enabling adaptive scheduling with minimal inference delay. [25], [26] further enhance system robustness in volatile networks by introducing temporal fairness metrics and topology-aware selection strategies, ensuring stable and efficient collaborative learning even under intermittent connectivity or extreme mobility.

Despite these advances, the process of client selection and the subsequent uploading of model updates may still expose sensitive client information, posing serious privacy risks in federated learning systems. In particular, the exchange of model updates during aggregation can lead to indirect leakage of local data characteristics or client behaviors, especially when adversaries or curious servers attempt to infer original data from shared gradients or parameters. To address these challenges, an emerging line of research introduces rigorous

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

TABLE I: Related Work Comparison

Work	Client Selection	Resource Scheduling	Decision Method	Privacy Protection	Privacy Setting Basis	Protection-Scheduling Integration
[13], [14], [15], [19]	✓	✓	DRL	✗	✗	✗
[17]	✓	✗	Optimization	✗	✗	✗
[18], [20], [26]	✓	✓	Optimization	✗	✗	✗
[21]	✓	✓	Auction	✗	✗	✗
[22]	✓	✗	Strategic selection	✗	✗	✗
[23]	✓	✗	OMP	SMPC	✗	✓
[24]	✓	✗	Bandit	✗	✗	✗
[25]	✓	✗	Lyapunov	✗	✗	✗
[27]	✓	✗	Optimization	DP	Client requirement	✓
[28]	✓	✗	Sampling	DP	Uniform	✓
[29]	✗	✗	DRL	DP	Privacy loss + accuracy	✗
[30]	✓	✓	DRL	DP	User level	✓
[31]	✗	✗	SRS	DP	Group preference	✗
[32]	✓	✗	MCDM	ZT	✗	✓
[33]	✓	✓	Lyapunov	DP	Privacy constraint	✓
This Paper	✓	✓	DRL	DP	Task priority	✓

Note: Privacy Setting Basis denotes the basis for privacy-level or privacy-budget setting, and Protection-Scheduling Integration denotes whether privacy or security protection is integrated into client selection or resource scheduling. SRS=sampling, randomizing and shuffling; DRL=deep reinforcement learning; DP=differential privacy; SMPC=secure multi-party computation; OMP=orthogonal matching pursuit; MCDM=multi-criteria decision making; ZT=zero trust.

privacy-preserving mechanisms, most notably differential privacy, to provide formal guarantees against information leakage during client selection and the uploading of model updates.

B. Privacy Protection in Client Selection

Although FL avoids direct raw-data sharing, uploaded model updates and gradients may still expose the system to indirect privacy threats, such as membership inference and gradient inversion attacks [27]. To address these risks, DP has become a mainstream privacy-preserving technique for FL, with formal foundations established in [34].

Most existing DP-FL methods adopt a uniform privacy budget for implementation simplicity, where all clients are assigned the same noise level [28], [35]. However, a uniform privacy budget is difficult to adapt to heterogeneous clients and tasks, since the same perturbation strength may provide insufficient protection for sensitive updates while introducing unnecessary noise for less sensitive ones. To address the limitation, personalized and heterogeneous DP mechanisms have been introduced into FL [29], [30], [31]. Specifically, Lu et al. [29] formulated personalized DP as a risk-aware perturbation policy optimization problem, where the perturbation mechanism, privacy budget, and relaxation parameter are selected according to privacy loss, model accuracy, and attack risk. Chen et al. [30] studied personalized DP in MEC-based multi-FL, where user privacy levels and edge resource constraints are jointly considered for UE selection and model utility maximization. In hierarchical FL, Chen et al. [31] proposed group local DP and a sampling-randomizing-shuffling mechanism to support group-level and client-level privacy personalization.

Besides privacy budget personalization, recent studies have further integrated privacy protection with client selection strategies [27], [28], [32], [33]. Chen et al. [28] combined importance client sampling with local Gaussian perturbation and tCDP-based privacy accounting to improve convergence under non-IID data while preserving privacy. Ma et al. [27] leveraged DP-induced privacy costs to guide client selection, reducing the participation probability of clients with higher privacy costs. Tahir et al. [32] incorporated privacy risk levels into a multi-criteria selection framework under a zero-trust architecture. Liu et al. [33] jointly optimized power allocation and client selection using Stackelberg game theory, achieving a trade-off between privacy and efficiency in healthcare IoT.

Although these studies improve privacy protection in FL from the perspectives of participant preference, training feedback, attack risk, heterogeneous privacy requirement, privacy cost, and group-level privacy preference, they still pay limited attention to task switching in VEC. In practical vehicular scenarios, the same vehicle may participate in FL while handling different task types across time slots, and the privacy demand of uploaded model updates may vary accordingly. As a result, privacy budgets determined only by participant preference, training feedback, attack risk, or privacy cost may still fail to match task-dependent privacy requirements. To address this issue, this paper determines the privacy configuration according to the priority of the current vehicular task and uses the configuration to calibrate the perturbation strength of uploaded model updates.

III. System Model

In this section, we first classify vehicular tasks based on their latency and privacy requirements, then present a priority-conditioned personalized differential privacy model for uploaded model updates. We further describe the communication and computation models, and finally formulate the joint client selection and resource allocation problem to maximize system utility with priority-conditioned privacy protection for uploaded model updates.

A. Model Review

We consider a system network model composed of a cloud center, multiple ES, and vehicular devices. The vehicular

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

devices are indexed as $\mathbb{N} = \{1, 2, ..., N\}$. Vehicles move along road networks and are equipped with limited computation and storage resources for executing locally generated tasks. The cloud center is denoted by C. The set of ES is represented as $\mathbb{E} = \{1, 2, ..., E\}$. We denote an adjacent ES as e'. ES also have certain computing capabilities. As illustrated in Fig. 1, each vehicular device maintains a model for local training. Driven by the continuous expansion of model scale, the computational demand of training sometimes exceeds the capacity of in-vehicle resources. To address the limitation, vehicles upload model updates to an ES e or the cloud C, leveraging the abundant computational resources of these infrastructures for model training, while strictly ensuring that the raw data remain on the device to protect user privacy. When

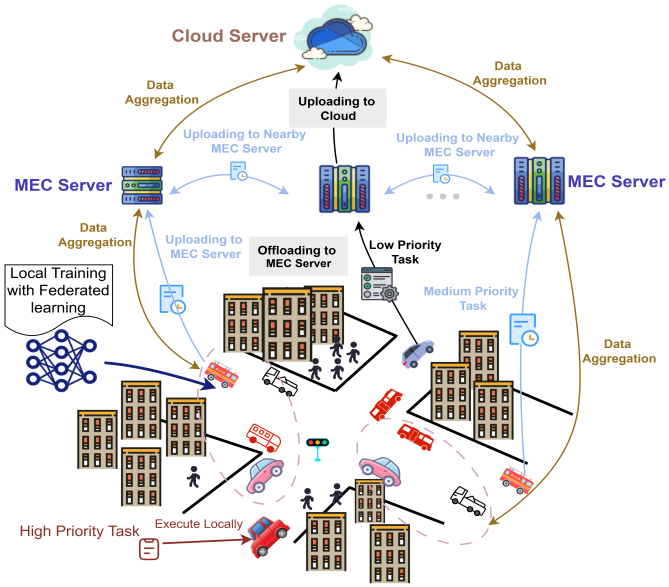


Fig. 1: Scenario.

vehicles are in operation, users access a variety of in-vehicle applications, which generate different types of requirements. $A_{n,t} = \{D_{n,t}, V_{n,t}, T_{n,t}^{max}\}$ represents the task generated by vehicle n at time t, where $D_{n,t}, V_{n,t}$ and $T_{n,t}^{max}$ denote the size of the task input, the required computing resources, and the maximum acceptable delay, respectively. Consequently, tasks are stratified into three distinct categories based on their model complexity and latency sensitivity, with Thr_1 and Thr_2 serving as the respective latency thresholds.

- High-priority tasks $A^h = \{D^h, V^h, T^{max,h}\}$: tasks such as emergency collision warnings and emergency braking functions provided by vehicles. These functions impose stringent latency and privacy requirements, and typically involve low complexity models with small input sizes. Consequently, they are well-suited for execution locally. The maximum acceptable delay $T^{max,h}$ should be less than Thr_1.
- Medium-priority tasks $A^m = \{D^m, V^m, T^{max,m}\}$: tasks such as adaptive cruise control, map navigation, lane keeping, and other driver assistance applications. Such tasks exhibit moderate model complexity and medium scale input size, with relatively lower latency and privacy requirements. Therefore, they are well-suited for transmitting model updates to an ES to enable collaborative training. The maximum acceptable delay $T^{max,m}$ should lie between Thr_1 and Thr_2.
- Low-priority tasks $A^l = \{D^l, V^l, T^{max,l}\}$: the type of tasks mainly includes multimedia and other entertainment applications. These tasks involve massive input data and require training with highly complex models, placing computational burdens beyond the capacity of individual vehicles. Consequently, they are best handled by transmitting model updates to the cloud, which offers scalable infrastructure capable of handling such workloads. The maximum acceptable delay $T^{max,l}$ should exceed Thr_2.

Given the heterogeneous requirements of vehicular tasks, the overall workflow can be described as follows. During vehicle operation, different applications continuously generate tasks, and these tasks differ in delay requirements, computation demands, and privacy sensitivity. Each vehicle has limited computation capability, and the available wireless and edge resources cannot support all vehicles in collaborative training at the same time. Therefore, the system needs to select appropriate vehicles in each time slot and allocate communication and computation resources to them. Each selected vehicle first trains the local model with its private data, while the raw data remain on the vehicle. According to the current decision, the generated model updates are then transmitted through an ES, an adjacent ES, or the cloud center. Although federated learning avoids direct raw data transmission, the uploaded model updates may still carry sensitive information related to the current task. Therefore, privacy protection according to task priority is applied to the uploaded updates. After receiving the protected model updates, the server performs aggregation and uses the updated global model for the next round of collaborative training.

B. Privacy Model

DP is a mechanism designed to protect the privacy of individual data entries while ensuring that the data remains useful and accessible. The mechanism introduces a controlled amount of random noise into the data processing, providing a strong guarantee that it is difficult for an adversary to determine whether a particular individual's data is included in the dataset. As referenced in [34], DP is defined as follows:

Definition 1 (ε-DP): a randomized algorithm $\mathcal{M}$ is introduced to satisfy ε-Differential Privacy if for all pairs of neighboring datasets $\mathcal{D}$ and $\mathcal{D}'$, where $\mathcal{D}$ and $\mathcal{D}'$ differ by at most one element, and for any subset $S \subseteq Range(\mathcal{M})$ of possible outputs, the following condition holds:

$$\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(\mathcal{D}') \in S], \tag{1}$$

where ε represents the privacy budget, which determines the degree of privacy protection. The smaller ε is, the higher the degree of privacy protection is.

Definition 2 (Local Differential Privacy): if the random algorithm $\mathcal{M}$ satisfies Eq. (2) for any two records d and d' in the dataset, then the algorithm $\mathcal{M}$ is said to satisfy ε-LDP:

$$\Pr[\mathcal{M}(d) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(d') \in S], \tag{2}$$

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

where ε denotes the privacy budget, and the LDP mechanism guarantees that the random algorithm $\mathcal{M}$ adheres to localized differential privacy by regulating the similarity between the outputs for any two records. Specifically, when the output is s, the adversary cannot statistically distinguish whether the input is d or d'.

Traditional differential privacy applies a uniform privacy protection level, which may fail to adequately protect sensitive data or overprotect non-sensitive data. Therefore, we introduce a Personalized Differential Privacy (PDP) mechanism, which customizes privacy protection based on diverse privacy sensitivities. The concept of privacy configuration is introduced to capture the varying importance and protection requirements of tasks regarding privacy. It plays a key role in setting the privacy budget and helps tasks establish privacy preferences that align with their specific needs, thus achieving an effective balance between privacy protection and data usability.

TABLE II: Symbolic Definitions

Notation	Description
System Notations	
$\mathbb{E}$	Set of edge servers
$\mathbb{N}$	Set of vehicles
C	Cloud center
Privacy Notations	
$\mathcal{D}, \mathcal{D}'$	Dataset and its adjacent dataset
$\mathcal{M}$	A random mechanism
$\xi_{n,t}$	The privacy configuration of vehicle n at time t
Network Notations	
$r^{e}_{n,k}$	Uplink rate of sub-channel k for vehicle n
r^{e}_{n}	Total uplink rate between vehicle n and ES e
$r^{e,down}_{n,k}$	Downlink rate of sub-channel k for vehicle n
$r^{e,down}_{n}$	Total downlink rate between vehicle n and ES e
$r^{e'}_{e,k}$	Uplink rate of sub-channel k for ES e
$r^{e'}_{e}$	Total uplink rate between ES e and nearby ES e'
r^{c}_{e}	Transmission rate between ES e and cloud c
$C^{SR}_{e}, C^{SR}_{e2e}, C^{SR}_{n2n}$	Spectrum resources
K^{e}, K^{e2e}, K^{n2n}	Total number of channels
$B^{e}, B^{e2e}, B^{n2n}, B^{e,down}$	Bandwidth of each sub-channel
$g^{e}_{n,k}, g^{e'}_{e,k}, g^{e,down}_{n,k}$	Channel gain of sub-channel k
$I^{e}_{n,k}, I^{e'}_{e,k}, I^{e,down}_{n,k}$	Interference of sub-channel k
$P^{e}_{n}, P^{e'}_{e}, P^{c}_{e}$	Power on transmission
σ	Noise power

Definition 3 (Privacy Configuration): the privacy configuration $\xi_{n,t}$ represents the required protection level for the uploaded model updates associated with the task performed by vehicle n at time slot t. Consequently, the corresponding privacy budget is given by:

$$\varepsilon_{n,t} = g(\xi_{n,t}), \quad g'(\xi) > 0, \tag{3}$$

where $g(\cdot)$ is a monotone calibration function. A larger $\xi_{n,t}$ induces a larger privacy budget and weaker privacy protection, while a smaller $\xi_{n,t}$ leads to a smaller privacy budget and stronger privacy protection. For the three task categories A^h, A^m, and A^l, the assigned privacy configurations satisfy $\xi^h < \xi^m < \xi^l$, indicating that tasks with higher priority are assigned smaller privacy configurations and therefore receive stronger privacy protection. Moreover, $\xi_{n,t}$ is constrained within the range $[1, 10]$.

Definition 4 ($\varepsilon_{n,t}$-Personalized Local Differential Privacy): for the privacy budget $\varepsilon_{n,t}$ induced by the privacy configuration $\xi_{n,t}$, if the randomized algorithm $\mathcal{M}$ produces the same output d^* for any two records d and d', as described by Eq. (4), then the algorithm $\mathcal{M}$ is considered to satisfy the criteria for $\varepsilon_{n,t}$-Personalized Local Differential Privacy ($\varepsilon_{n,t}$-PLDP).

$$\Pr[\mathcal{M}(d) = d^*] \leq e^{\varepsilon_{n,t}} \cdot \Pr[\mathcal{M}(d') = d^*]. \tag{4}$$

Definition 5 ($\ell_1 - Sensitivity$): assume that there exists a function $\mathcal{F} : \mathcal{D} \to \mathbb{R}^{\mathcal{D}}$ with the ℓ_1 -sensitivity definition shown in Eq. (5).

$$\Delta\mathcal{F} = \max_{\substack{\mathcal{D}\sim\mathcal{D}' \\ \|\mathcal{D}-\mathcal{D}'\|_1=1}} \|\mathcal{F}(\mathcal{D}) - \mathcal{F}(\mathcal{D}')\|_1, \tag{5}$$

where $\|\cdot\|_1$ denotes the ℓ_1-norm, the ℓ_1-sensitivity of the function $\mathcal{F}$ measures the impact of adding or removing a single record on the function's output. Specifically, it represents the maximum difference in the function's output when the input datasets are adjacent. The sensitivity is an intrinsic property of the function and remains independent of variations in the input datasets.

Definition 6 (Laplace Mechanism): given a query function $\mathcal{F} : \mathcal{D} \to \mathbb{R}^D$, the Laplace Mechanism outputs $\mathcal{M} = \mathcal{F}(\mathcal{D}) + Lap(\frac{\Delta\mathcal{F}}{\varepsilon})$. The added noise follows the Laplace distribution characterized by the probability density function:

$$\mathrm{Lap}(x \mid b) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right), \tag{6}$$

where $\Delta\mathcal{F} = \max_{\mathcal{D},\mathcal{D}'} \|\mathcal{F}(\mathcal{D}) - \mathcal{F}(\mathcal{D}')\|_1$ is the global sensitivity, the Laplace Mechanism utilizes ℓ_1-sensitivity, making it particularly well-suited for the federated learning training process. Here, b is the scale parameter of the Laplace distribution, and ε represents the privacy budget. According to Definition 3, the privacy budget used for the clipped uploaded model updates is specified as $\varepsilon_{n,t} = g(\xi_{n,t})$. Consequently, the perturbation scale for vehicle n at time slot t is given by $b_{n,t} = \frac{\Delta F}{\varepsilon_{n,t}} = \frac{\Delta F}{g(\xi_{n,t})}$.

For a clipped uploaded update, the strength of the Laplace perturbation can be measured by the variance of the perturbation applied to each parameter component. For the model update uploaded by vehicle n at time slot t, the perturbation variance is given by:

$$2b^2_{n,t} = 2\left(\frac{\Delta F}{\varepsilon_{n,t}}\right)^2 = 2\left(\frac{\Delta F}{g(\xi_{n,t})}\right)^2. \tag{7}$$

According to Definition 3, a higher task priority corresponds to a smaller privacy budget and a larger perturbation scale. Therefore, the perturbation scales of the three task categories satisfy $b^h > b^m > b^l$, and the corresponding perturbation variances satisfy $2(b^h)^2 > 2(b^m)^2 > 2(b^l)^2$. A smaller privacy budget

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

increases the perturbation variance in the uploaded update, thereby enhancing privacy protection but causing a larger loss in model utility. In contrast, a larger privacy budget reduces the variance and weakens privacy protection but preserves more useful model update information. During aggregation, the server performs weighted aggregation on the noisy updates according to the local data proportion of each selected vehicle. Thus, the local perturbations enter the aggregated update with the same weights. Since the selected vehicles generate Laplace perturbations independently, the perturbation variance in the aggregated update for the selected vehicle set $\mathcal{K}_t$ is given by:

$$2\sum_{n\in\mathcal{K}_t}\left(\frac{|D_n|}{\sum_{i\in\mathcal{K}_t}|D_i|}\right)^2 b_{n,t}^2, \tag{8}$$

where $\frac{|D_n|}{\sum_{i\in\mathcal{K}_t}|D_i|}$ represents the aggregation weight determined by the local data size of vehicle n. The perturbation variance in the aggregated update is jointly determined by the perturbation scale and the weight of each selected vehicle. Consequently, the effect of the privacy budget on local perturbation strength is further propagated to the global update and affects the utility of the global model.

Eq. (8) characterizes the propagation of the perturbation variance in a single round to the aggregated update. For multi-round FL training, repeated vehicle participation causes privacy loss to accumulate across training rounds. According to [34], the cumulative privacy budget of vehicle n over T rounds can be expressed as:

$$\varepsilon_n^{\text{cum}} = \sum_{t=1}^{T}\mathbb{I}(n\in\mathcal{K}_t)\varepsilon_{n,t} = \sum_{t=1}^{T} I(n\in\mathcal{K}_t)g(\xi_{n,t}), \tag{9}$$

where $\mathbb{I}(\cdot)$ is an indicator function, which returns 1 if the specified condition is satisfied and 0 otherwise. Since $g'(\xi) > 0$, a smaller privacy configuration $\xi_{n,t}$ induces a smaller privacy budget $\varepsilon_{n,t}$. Consequently, vehicles associated with tasks of higher priority not only receive stronger perturbation in each selected round, but also slow the growth of the cumulative privacy budget over repeated participation.

C. Latency Model

The latency model consists of two parts: transmission latency and computational latency.

1) Transmission Latency: the transmission latency represents the time spent transferring model updates and returned results among vehicles, ESs, and the cloud center. This latency is fundamentally determined by the transmission rates over the selected communication path. Based on the Shannon formula, the uplink transmission rate of a specific sub-channel k from vehicle n to ES e is defined as:

$$r_{n,k}^e = B^e \log_2\left(1+\frac{P_n^e\cdot g_{n,k}^e}{\sigma^2+I_{n,k}^e}\right), \tag{10}$$

where B^e is the sub-channel bandwidth, P_n^e is the vehicle's transmission power, $g_{n,k}^e$ is the channel gain, σ^2 is the noise power, and $I_{n,k}^e$ is the inter-channel interference. The total uplink rate for vehicle n to ES e allocated by the binary variable $y_{n,k}^e$ is defined as:

$$r_n^e = \sum_{k\in\mathbb{K}^e} y_{n,k}^e \cdot r_{n,k}^e. \tag{11}$$

The rates for other links are modeled analogously. Specifically, we define $r_n^{e,down}$ as the total downlink rate to vehicle n, and $r_e^{e'}$ as the total transmission rate from an ES e to an adjacent ES e'. They are expressed as:

$$r_n^{e,down} = \sum_{k\in\mathbb{K}^e} y_{n,k}^{e,down}\cdot r_{n,k}^{e,down}, \tag{12}$$

$$r_e^{e'} = \sum_{k\in\mathbb{K}^{e2e}} y_{e,k}^{e'}\cdot r_{e,k}^{e'}. \tag{13}$$

The ES is connected to the cloud center via optical fiber, offering stable and high-speed transmission. Consequently, the transmission rate is modeled as a fixed rate r_e^c.

The total communication latency for an assistance path is the duration of its round-trip, which consists of uploading model updates and downloading the returned global model parameters. When transmitting the updates to an ES, the transmission latency is expressed as:

$$T_{n,e}^{trans} = \frac{D_n}{r_n^e} + \frac{D_n}{r_n^{e,down}}. \tag{14}$$

When the ES e sometimes cannot meet the computational requirements, it is necessary to forward the associated updates to an adjacent ES e', and the transmission latency can be expressed as:

$$T_{n,e,e'}^{trans} = \frac{D_n}{r_n^e} + \frac{D_n}{r_e^{e'}} + \frac{D_n}{r_e^{e'}} + \frac{D_n}{r_n^{e,down}}. \tag{15}$$

For uploading updates to the cloud, the transmission latency includes uploading to the edge server, sending to the cloud, and downloading results back to the vehicle. Similar to the adjacent edge server case, the cloud communication also involves a round trip for the exchange of model updates between the edge server and cloud, and the transmission latency can be denoted as:

$$T_{n,c}^{trans} = \frac{D_n}{r_n^e} + \frac{D_n}{r_e^c} + \frac{D_n}{r_e^c} + \frac{D_n}{r_n^{e,down}}. \tag{16}$$

The overall transmission latency is determined by the selected communication path, with binary indicator variables η_e, $\eta_{e'}$, and η_c representing the choice of transmitting to the ES, to an adjacent ES or to the cloud, respectively. Consequently, the total transmission latency is expressed as:

$$T_{n,t}^{trans} = \eta_e T_{n,e}^{trans} + \eta_{e'} T_{n,e,e'}^{trans} + \eta_c T_{n,c}^{trans}. \tag{17}$$

2) Computation Latency: the computation latency defines the time required for data processing at the various nodes capable of providing computation.

The local computation path represents a scenario where all processing is handled by the vehicle's onboard unit. For this path, the computation latency is determined by the computing capability f_n of vehicle n expressed as:

$$T_{loc}^{comp} = \frac{V_n}{f_n}. \tag{18}$$

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

For paths that involve external assistance from a server, the processing time at the server is a key component. The computation latency on an ES e depends on its total capability f_e and the proportion of allocated ι_n^e, denoted as:

$$T_{n,e}^{comp} = \frac{V_n}{\iota_n^e f_e}. \tag{19}$$

An analogous calculation, using the appropriate resource allocation ι_e^e and the server capability $f_{e'}$, determines the computation latency on an adjacent ES e', which can be denoted as:

$$T_{n,e'}^{comp} = \frac{V_n}{\iota_e^{e'} f_{e'}}. \tag{20}$$

For the cloud center, the computation latency is determined by the cloud computing capability f_c, which is expressed as:

$$T_{n,c}^{comp} = \frac{V_n}{f_c}. \tag{21}$$

In summary, the total computation latency can be expressed as the following:

$$T_{n,t}^{comp} = \eta_l T_{loc}^{comp} + \eta_e T_{n,e}^{comp} + \eta_{e'} T_{n,e'}^{comp} + \eta_c T_{n,c}^{comp}, \tag{22}$$

where η_l denotes whether local computation is selected.

3) Total Latency: finally, the overall latency can be expressed as:

$$T_{n,t} = T_{n,t}^{trans} + T_{n,t}^{comp}. \tag{23}$$

D. Energy Model

1) Transmission Energy: the transmission energy is the energy consumed by communication modules during the transfer of model updates within the assistance loops.

When updates are transmitted to an ES e, the transmission energy includes the energy for uploading the updates and for downloading the enhanced version. The round-trip transmission energy from the ES e to vehicle n can be expressed as:

$$E_{n,e}^{trans} = \frac{P_n^e D_n}{r_n^e} + \frac{P_n^{e,down} D_n}{r_n^{e,down}}. \tag{24}$$

The energy consumption increases when the updates are transmitted to an adjacent ES e', which must account for the power consumed during the whole trip. Consequently, it can be expressed as:

$$E_{n,e,e'}^{trans} = \frac{P_n^e D_n}{r_n^e} + \frac{P_e^{e'} D_n}{r_e^{e'}} + \frac{P_e^{e',down} D_n}{r_e^{e',down}} + \frac{P_n^{e,down} D_n}{r_n^{e,down}}. \tag{25}$$

Similarly, the transmission energy for the cloud path, $E_{n,c}^{trans}$, is also composed of the energy for the whole trip, which can be expressed as:

$$E_{n,c}^{trans} = \frac{P_n^e D_n}{r_n^e} + \frac{P_e^c D_n}{r_e^c} + \frac{P_e^{c,down} D_n}{r_e^{c,down}} + \frac{P_n^{e,down} D_n}{r_n^{e,down}}. \tag{26}$$

Consequently, the total transmission energy consumption is given by:

$$E_{n,t}^{trans} = \eta_e E_{n,e}^{trans} + \eta_{e'} E_{n,e,e'}^{trans} + \eta_c E_{n,c}^{trans}. \tag{27}$$

2) Computation Energy: Computation energy is the energy consumed by the processing units at the various nodes capable of executing the task or providing assistance.

Based on the research by [36], [37], the energy consumption of local execution is proportional to the square of the CPU voltage. It is also observed that the CPU clock frequency f_n is approximately linearly proportional to the supply voltage. Hence, the energy consumption for a single CPU cycle can be expressed as $\zeta_n (f_n)^2$, where the value of ζ is contingent upon the CPU architecture of vehicle n. Consequently, the local computation energy is expressed as:

$$E_{loc}^{comp} = \zeta_n (f_n)^2 V_n. \tag{28}$$

When the workload is transmitted to an ES e, it leads to a different energy consumption profile. The energy consumed on the path is modeled as a linear function of the workload and the ES's specific energy efficiency per unit of computation Z_e, which can be expressed as:

$$E_{n,e}^{comp} = V_n Z_e. \tag{29}$$

The principle also applies to the adjacent ES path, where the energy consumption is calculated analogously using its own energy coefficient $Z_{e'}$, expressed as:

$$E_{n,e'}^{comp} = V_n Z_{e'}. \tag{30}$$

For the cloud path, the computation energy is calculated via the cloud energy coefficient Z_c, which can be denoted as:

$$E_{n,c}^{comp} = V_n Z_c. \tag{31}$$

Consequently, the computation energy can be expressed as:

$$E_{n,t}^{comp} = \eta_l E_{loc}^{comp} + \eta_e E_{n,e}^{comp} + \eta_{e'} E_{n,e'}^{comp} + \eta_c E_{n,c}^{comp}. \tag{32}$$

3) Total Energy: finally, the overall energy consumption can be expressed as:

$$E_{n,t} = E_{n,t}^{trans} + E_{n,t}^{comp}. \tag{33}$$

E. Problem Formulation

With the development of VEC, various tasks generated by applications can be executed through multiple methods. In resource-constrained IoV environments, the ES, acting as both a scheduler and a compute resource, coordinates workload offloading, manages resource allocation, and participates in collaborative model training and aggregation. To enhance user experience and maximize system processing efficiency under limited resources and varying task requirements, the optimization problem is formulated as follows:

$$\max Q = \frac{1}{T} \sum_{t=1}^{T} \left\{ \sum_{n \in N} \frac{F_{n,t}}{\lambda T_{n,t} + (1-\lambda) E_{n,t}} + \phi acc_t \right\}. \tag{34}$$

$$\text{s.t.} \quad \eta_l, \eta_e, \eta_{e'}, \eta_c \in \{0,1\}, \tag{34a}$$

$$\eta_l + \eta_e + \eta_{e'} + \eta_c \leq 1, \tag{34b}$$

$$\iota \in [0,1], \tag{34c}$$

$$\sum_{k \in K^e} y_{n,k}^e \leq 1, \quad \sum_{k \in K^{e2e}} y_{e,k}^{e'} \leq 1, \tag{34d}$$

$$T_{n,t} \leq T_{n,t}^{\max}. \tag{34e}$$

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

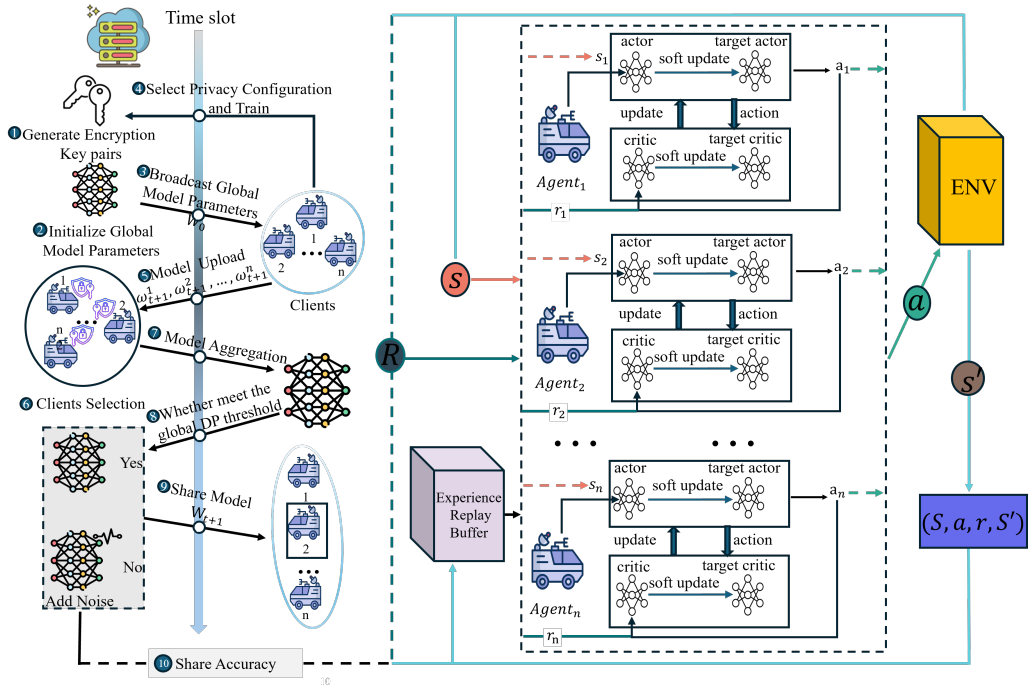


Fig. 2: Algorithm framework.

where $F_{n,t}$ represents the number of tasks completed by vehicle n, λ is the weighting coefficient between latency and energy consumption, ϕ is the accuracy-scaling factor, and acc_t denotes the global accuracy at time slot t. Constraint (34a) specifies the feasible task execution modes. The accuracy acc_t measures the performance of the model on the test dataset; unlike the loss, it provides a more direct and interpretable indicator. Constraint (34b) denotes that a vehicle can select at most one execution method for a given task. Constraint (34c) ensures that the computational resource ratio allocated to the task remains within the feasible range. Constraint (34d) denotes that during communication between devices, each device can be assigned at most one channel in each scheduling period. Constraint (34e) denotes that the task execution time must meet the maximum acceptable delay for the task.

IV. Approach

To tackle the joint optimization under dynamic vehicular environments, we formulate the problem as an MDP and design a multi-agent DRL framework. Complementing this, we also establish a federated learning architecture enhanced with a task-priority-based personalized differential privacy mechanism to safeguard uploaded model updates during collaborative training.

A. DRL-Based Decision

In the VEC environment, as the number of connected devices grows, the increasing task workload imposes significant pressure on system resources, rendering the efficient distribution of communication and computing resources a complex challenge. To tackle the issue, the optimization problem is converted into an MDP, which is represented by the tuple $(\mathcal{S}, \mathcal{A}, r, \gamma)$.

1) State Space: at time slot t, the state information observed by ES e can be represented as:

$$S_{e,t} = \left(A_{e,t}, Q_{e,t}, f_{e,t}, C_{e,t}^{SR}\right), \tag{35}$$

where $A_{e,t} = (A_{1,t}, A_{2,t}, \ldots, A_{N,t})$ represents the task state of the vehicles connected to e and $Q_{e,t}$ denotes the client-value vector for associated vehicles. $f_{e,t}$ denotes the available computing resources and $C_{e,t}^{SR}$ indicates the available spectrum resources. The system state space can be defined as $s_t = (S_{1,t}, S_{2,t}, \ldots, S_{E,t})$.

2) Action Space: at time slot t, the action space observed by ES e is expressed as:

$$a_{e,t} = (\eta_l, \eta_e, \eta_{e'}, \eta_c, y_{n,k}^{e}, y_{e,k}^{e'}, \iota), \tag{36}$$

where $\eta_e, \eta_{e'}, \eta_c$ jointly determine whether vehicle n uploads its model updates to ES e, to an adjacent ES e', or to the cloud center in this slot with required communication resources $y_{n,k}^{e}, y_{e,k}^{e'}$ and the corresponding computational resources ι for the selected path. Specifically, $\eta_l = 1$ indicates local execution without uploading, $\eta_e = 1$ indicates that vehicle n uploads its model updates to ES e and is selected for federated learning, $\eta_{e'}$ indicates uploading to an adjacent ES e', and η_c indicates uploading to the cloud center C. The system action space can be defined as $a_t = (a_{1,t}, a_{2,t}, \ldots a_{E,t})$.

3) Reward Function: to address the multi-objective optimization problem effectively, a hierarchical reward function is designed to jointly optimize task execution, system overhead, and learning performance. Specifically, the reward consists of delay, energy consumption, feasibility constraint, and model accuracy, where the delay and energy rewards are normalized to comparable bounded ranges. The overall reward is expressed as:

$$r_{e,t} = \nu_d r_{e,t}^{delay} + \nu_e r_{e,t}^{energy} + \nu_c r_{e,t}^{constraint} + \phi r_t^{acc}, \tag{37}$$

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

where ν_d, ν_e, and ν_c denote the weights of the delay, energy, and constraint rewards, respectively, and $\nu_d + \nu_e + \nu_c = 1$. The constituent components are described as follows:

- Delay Reward Component: penalizes execution delays exceeding the maximum threshold:

$$r_{e,t}^{delay} = \begin{cases} 0, & T_{n,t} \leq T_{n,t}^{max} \\ -\alpha_1(T_{n,t} - T_{n,t}^{max}), & T_{n,t} \geq T_{n,t}^{max} \end{cases} \quad (38)$$

  where α_1 is the penalty coefficient for delay violation.
- Energy Reward Component: penalizes unnecessary energy consumption:

$$r_{e,t}^{energy} = -\alpha_2 E_{n,t}, \quad (39)$$

  where $E_{n,t}$ denotes the energy consumption, and α_2 is the penalty coefficient for energy consumption.
- Constraint Reward Component: ensures effective utilization of actions and resources:

$$\begin{aligned} r_{e,t}^{constraint} = & -\mathbb{I}(\iota \notin [0,1]) \\ & -(\eta_l + \eta_e + \eta_{e'} + \eta_c) \\ & \cdot \mathbb{I}\big[(\eta_l, \eta_e, \eta_{e'}, \eta_c) \notin \{0,1\}, \\ & \quad (\eta_l + \eta_e + \eta_{e'} + \eta_c \lneq 1)\big] \\ & -(\sum_{k \in \mathbb{K}^e} y_{n,k}^e) \cdot \mathbb{I}(\sum_{k \in \mathbb{K}^e} y_{n,k}^e \lneq 1) \\ & -(\sum_{k \in \mathbb{K}^{e2e}} y_{e,k}^{e'}) \cdot \mathbb{I}(\sum_{k \in \mathbb{K}^{e2e}} y_{e,k}^{e'} \lneq 1). \end{aligned} \quad (40)$$

B. FL_MADDPG Based Scheduling Policy

In dynamic environments, the device strategies evolve as training progresses, leading to intense competition for resources. We adopt FL_MADDPG based scheduling for the formulated MDP, while the priority-conditioned PDP mechanism is applied in the FL-PDP update procedure. Compared with DDPG, the FL_MADDPG based scheduling policy is learned through centralized training and applied through decentralized execution. During training, centralized learning allows agents to share global information, while during execution, agents rely only on local observations to make decisions. The design enables FL_MADDPG to outperform DDPG in handling complex interactions among multiple agents, offering better performance and stability.

FL_MADDPG effectively handles continuous action spaces through the actor-critic architecture with dual neural networks and experience replay. The actor network outputs an agent's actions based on the input state, while the critic network takes both state and action as input and outputs the Q-value of the current policy to evaluate its effectiveness. The experience replay mechanism stores the agent's state transitions in the environment and allows for random sampling from the replay buffer to train the model, which helps to break the correlation between data and stabilize the training process.

The pseudocode of the proposed FL_MADDPG based scheduling policy algorithm is shown in Algorithm 1, and deterministic policies of the participants are denoted as $\mu = \{\mu_{\theta_1}, \ldots, \mu_{\theta_E}\}$, and $\theta = \{\theta_1, \ldots, \theta_E\}$ represents the network parameters. Consequently, the loss function is denoted as:

$$L(\theta_e) = \mathbb{E}_{S,a,r,S'}\left[\left(Q_e^{\mu}(S, a_1, \ldots, a_E) - y\right)^2\right], \quad (41)$$

$$y = r_e + \gamma\, Q_e^{\mu'}(S', a_1', \ldots, a_E')\Big|_{a_j' = \mu_j'(S_j')}, \quad (42)$$

where γ represents the discount factor. The gradient of the deterministic policy for ES e can be expressed as:

$$\begin{aligned} \nabla_{\theta_e} J(\mu_e) = & \mathbb{E}_{S,a \sim D} \\ & \Big[\nabla_{\theta_e} \mu_e(a_e \mid s_e)\, \nabla_{a_e} Q_e^{\mu}(S, a_1, \ldots, a_E)\big|_{a_e = \mu_e(s_e)}\Big], \end{aligned} \quad (43)$$

where $Q_e^{\mu}(S, a_1', \ldots, a_E')$ is the Q-value function. The actor network is updated through the process of gradient descent, which can be expressed as:

$$\nabla_{\theta_e} J \approx \frac{1}{X} \sum_i \nabla_{\theta_e} \mu_e(s_e^i) \nabla_{a_e} Q_e^{\mu}(s^i, a_1^i, \ldots a_E^i)\Big|_{a_e = \mu_e(s_e^i)}, \quad (44)$$

where X denotes the mini-batch size, and i denotes the index of samples.

Algorithm 1 FL_MADDPG based scheduling policy

Input: Critic network Q and actor network μ, target network Q' and μ', environment env
Output: Optimal parameters $\theta_{actor}^*, \theta_{critic}^*$ and optimal policy $\pi^*(s_i)$
Initialize:
Initialize the weights of Actor and Critic networks, the replay buffer M
for episode = 1, 2, … **do**
  Receive initial state s
  Initialize a random process $\mathcal{N}$ for action exploration
  for t=1 to max-episode-length **do**
    Each agent e selects an action $a_{e,t} = \mu_e(s_{e,t}) + \mathcal{N}_t$ according to the current policy
    Execute a_t and observe reward r_t and next state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) into replay buffer M
    for agent $e = 1, 2, \ldots$ **do**
      Randomly sample a mini-batch from M
      Update the critic network by minimizing the loss function according to Eq. (41) and Eq. (42)
      Update the actor network with the sampled policy gradient according to Eq. (44)
    end for
    Update target network parameters for each agent e
    $\theta_e' = \tau\theta_e + (1-\tau)\theta_e'$
  end for
end for

C. FL-Based Training Framework with PDP

This section presents the detailed design of FL-PDP, including the implementation procedures and the personalized differential privacy perturbation algorithm. After the scheduling policy has been trained, subsequent FL rounds use the learned policy to determine client selection and resource allocation before the selected vehicles execute the FL-PDP update procedure. The overall framework of FL-PDP comprises four distinct stages, as illustrated in Fig. 2. The functions and objectives of each stage are described as follows:

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

1) Model distribution: devices selected according to the learned scheduling policy download the initial model parameters W_t, privacy configurations, and encryption keys from the server, which are then used for local training and model perturbation.
2) Local training and protected update uploading: each selected vehicle n initializes the local model with W_t and trains the model using its local dataset to obtain local model parameters ω_{t+1}^n. After training, vehicle n computes its model update $\Delta\omega_t^n = \omega_{t+1}^n - W_t$, performs update clipping on $\Delta\omega_t^n$ to obtain $\Delta\bar{\omega}_t^n$ and then applies perturbation to $\Delta\bar{\omega}_t^n$ based on its respective privacy configuration. The perturbed model updates $\Delta\tilde{\omega}_t^n$ are then uploaded to the central server together with the encrypted privacy configurations.
3) Model aggregation: the server aggregates the perturbed model updates $\Delta\tilde{\omega}_t^j$ from the selected clients. The need for additional aggregation-level perturbation is evaluated by comparing the mean induced privacy budget $\bar{\varepsilon}$ with the global privacy budget threshold ε_{max}.
4) Model update: the devices download the global model parameters W_{t+1} from the central server for local training in the next round, repeating the above process until the model converges or the preset number of iterations is reached.

The detailed steps of the FL-PDP training are summarized in Algorithm 2, which covers the entire process and methods of model aggregation.

Algorithm 2 FL-PDP training procedure

Input: Initialized model parameters W_0, aggregation rounds T, global privacy budget threshold ε_{max}, public key $K_p = (e, \rho)$, private key $K_s = (d, \rho)$, learning rate lr, local batch size B, number of local epochs C, loss function L
Output: W_T
Broadcast W_0, privacy calibration function $g(\cdot)$, and K_p to all clients
for round $t = 0, 1, \ldots, T-1$ **do**
  Obtain the selected client set $\mathcal{K}_t$
  for each selected client n in $\mathcal{K}_t$ **do**
    $\omega_t^n \leftarrow W_t$
    for epoch $= 1, 2, \ldots, C$ **do**
      for each batch $b \in B$ **do**
        $\omega_t^n \leftarrow \omega_t^n - lr \cdot \nabla L(\omega_t^n; b)$
      end for
    end for
    $\omega_{t+1}^n \leftarrow \omega_t^n$
    $\Delta\omega_t^n \leftarrow \omega_{t+1}^n - W_t$
    Apply local noise: $\tilde{\Delta\omega_t^n} \leftarrow \text{Noisy}(\text{Clip}(\Delta\omega_t^n), b_{n,t})$
    $\xi_{n,t}^* \leftarrow (\xi_{n,t})^e \mod \rho$
    Upload $\tilde{\Delta\omega_t^n}$ and $\xi_{n,t}^*$ to server
  end for
  for each uploaded $\xi_{n,t}^*$ **do**
    $\varepsilon_{n,t} \leftarrow \xi_{n,t} \leftarrow (\xi_{n,t}^*)^d \mod \rho$
  end for
  Calculate the mean induced privacy budget $\bar{\varepsilon}$
  $W_{t+1} \leftarrow W_t + \sum_{j \in \mathcal{K}_t} \frac{\|D_j\|}{\sum_{i \in \mathcal{K}_t} \|D_i\|} \tilde{\Delta\omega_t^j}$
  if $\bar{\varepsilon} > \varepsilon_{max}$ **then**
    Apply global noise to the global model: $W_{t+1} \leftarrow \text{Noisy}(W_{t+1}, \varepsilon_{max})$
  end if
end for

V. Experiments

In this section, we implement the proposed FL_MADDPG with the FL-PDP mechanism using TensorFlow, focusing on the client selection and the impact of client privacy configurations on system performance.

A. Parameter and Dataset Settings

In this section, we implement the FL_MADDPG client selection and resource allocation algorithm in TensorFlow 2.9.0 and build a simulation environment with 10 devices, 3 ESs, and 1 cloud server. During FL_MADDPG training, the actor network consists of two fully connected hidden layers, while the critic network has three, both using ReLU activations. For federated learning, we employ a CNN with two convolutional layers of 16 and 32 filters respectively, each using a 5×5 kernel with stride 2. Local client training is performed for 3 epochs with a batch size of 128.

We evaluate our framework on MNIST, CIFAR-10, and LISA. The MNIST and CIFAR-10 datasets serve as standard image classification benchmarks to assess the framework under different data complexities, while LISA provides traffic-related visual samples. For each dataset, training samples are uniformly partitioned across ten vehicles, and each vehicle performs local training only on its assigned subset. Since the evaluated datasets provide class labels but no priority annotations, priority assignment is separated from dataset labels. Dataset samples are used for local model training and generating uploaded model updates. For each generated vehicular task, the task model assigns a priority category according to its service requirements, and the priority category further determines the corresponding privacy configuration. This setting allows the same dataset to be evaluated under different priority-conditioned privacy configurations. For benchmarking client selection and resource allocation, we compare against four baselines, namely PPO [38], DQN [39], DDPG [40], and MATD3 [41].

B. Experiment Results and Analysis

In this section, we evaluate the performance of the proposed algorithm under different environments by analyzing its convergence, task completion rate and energy consumption. We further assess its effectiveness in scenarios under personalized privacy protection. Given the stochasticity of DRL training and the dynamic vehicular environment, we independently repeated selected experiments three times.

1) Convergence Analysis: Fig. 3 illustrates the convergence performance of the five algorithms over 3000 training episodes. During the initial 1000 episodes, the average rewards exhibit high variance, as the agents operate in the exploration phase with insufficient data in the experience-replay buffer.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

TABLE III: Experimental Settings

Parameter Description	Value/Range
Network Parameters	
Thr_1	0.5 s
Thr_2	1.5 s
σ^2	-120 dB
Bandwidth of vehicle	1 GHz
Bandwidth of ES	100 MHz
Computing resources of vehicle	2 GHz
Computing resources of ES	16 GHz
Computing resources of cloud	30 GHz
Transmission power of vehicle	[0.1, 0.5] W
Transmission power of ES	[0.5, 1.0] W
Channel gain of vehicle	[-45, -20] dB
Channel gain of ES	[-45, -30] dB
CPU cycle frequency	[10, 20] MHz
Deep Reinforcement Learning Algorithm Parameters	
Buffer Size	10000
Mini-Batch Size	128
Learning rate of actor network	0.0003
Learning rate of critic network	0.001
ν_d, ν_e, ν_c	0.3, 0.3, 0.4
ϕ	0.16
λ	0.50
Federated Learning Algorithm Parameters	
High priority task privacy configuration	[1-3]
Medium priority task privacy configuration	[4-7]
Low priority task privacy configuration	[8-10]
Learning rate	0.001
Local epoch	3
Batch size	128
Noise type	Laplace

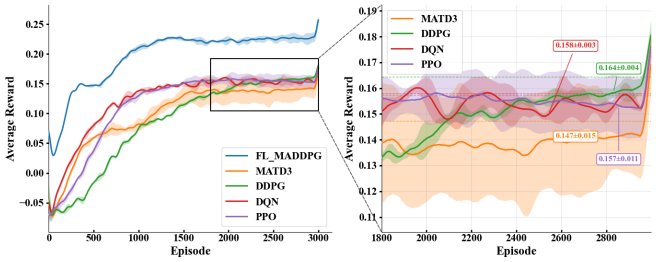


Fig. 3: Convergence of different algorithms.

As training progresses, all algorithms eventually converge to stable reward levels. Among the baselines, the MATD3 algorithm outperforms DDPG, DQN, and PPO, indicating that the twin-critic architecture effectively mitigates overestimation and enhances learning stability in multi-agent environments. Nevertheless, the FL_MADDPG framework attains the highest average reward upon convergence. The convergence advantage indicates that the centralized-training and decentralized-execution paradigm enables the FL_MADDPG framework to capture inter-agent interactions and global resource competition effectively, thereby yielding optimal client selection and resource allocation policies.

2) Energy Consumption Analysis: Fig. 4 compares the normalized energy consumption of FL_MADDPG, DDPG, DQN, PPO, and MATD3 under different device and ES scales. As shown in Fig. 4(a), the normalized energy consumption of all algorithms generally increases as the number of devices grows. This trend is primarily attributed to intensified competition for wireless channels and computation resources as more vehicles participate in the system, which raises transmission and execution costs. FL_MADDPG maintains the lowest energy consumption under most device configurations and exhibits a more gradual increase as the system scales. Fig. 4(b) further presents the energy consumption under different ES configurations. When the number of ESs is limited, FL_MADDPG demonstrates a clear energy advantage over the baseline algorithms. As the number of ESs increases, the energy gap among different algorithms gradually narrows, and the overall energy consumption does not follow a strictly monotonic pattern. Overall, FL_MADDPG maintains competitive energy efficiency under both device-scale expansion and ES-scale variation, demonstrating the effectiveness and stability of the energy-aware scheduling strategy in joint client selection and resource allocation.

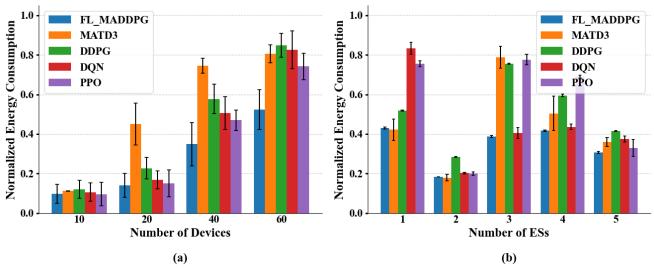


Fig. 4: Normalized energy consumption of different algorithms.

3) Task Completion Rate and Delay Analysis: Fig. 5 illustrates the task completion rate of the evaluated algorithms under varying communication and computation resource capacities. As the communication bandwidth and computation capacity increase, the task completion rates of all algorithms generally improve. Increased bandwidth alleviates transmission bottlenecks for uploaded model updates, while greater computation capacity reduces processing delays at selected execution nodes; together, these effects enable a higher proportion of tasks to satisfy their latency constraints. Under constrained settings, such as 40 MHz communication bandwidth and 8 GHz computation capacity, all algorithms exhibit lower completion ratios due to pronounced transmission and computation bottlenecks.

Fig. 6 further evaluates the average delay and task completion rate of different algorithms under varying device and edge server scales. As illustrated in Fig. 6(a), the growing number of devices intensifies competition for wireless channels and computation resources, leading to a general increase in the average delay across all algorithms. FL_MADDPG maintains the lowest average delay across all device scales. Fig. 6(b) shows that FL_MADDPG achieves a high task completion rate under small- and medium-scale device settings; however, this metric decreases when the number of devices reaches 60. This decrease is mainly attributable to the scheduling objective of FL_MADDPG. Specifically, FL_MADDPG learns

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

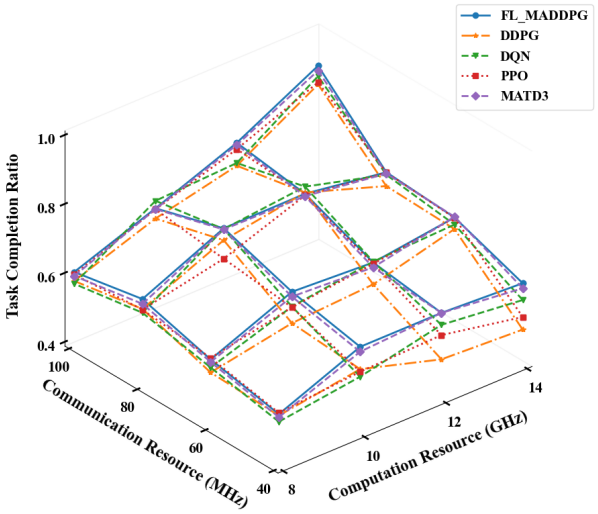


Fig. 5: Task completion rate of different algorithms under different communication and computing resource environments.

scheduling decisions from the global system state and jointly considers delay, energy consumption, resource constraints, and learning performance, rather than maximizing only the number of completed tasks. Therefore, under heavy-load scenarios, the learned policy tends to preserve overall system efficiency, although some tasks may still fail to meet their delay constraints. Fig. 6(c) and Fig. 6(d) further present the delay and task completion performance under different ES configurations. As the number of ESs increases, additional edge resources alleviate resource contention and improve service capacity, which generally reduces the average delay and correspondingly increases the task completion rate for all algorithms. When the ES scale further increases, the performance gain becomes less significant, indicating that the resource bottleneck is gradually weakened. Overall, FL_MADDPG shows a clear delay advantage and competitive task completion performance under both device-scale expansion and ES-scale variation, demonstrating the adaptability and delay-aware scheduling effectiveness of the proposed method.

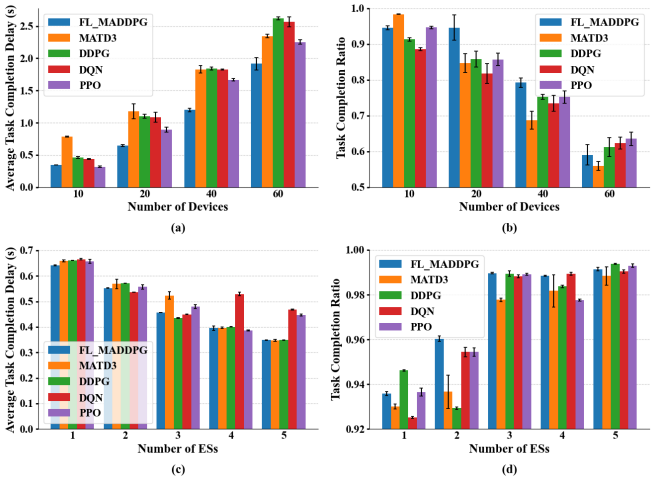


Fig. 6: The impact of different numbers of devices and ESs on delay and task completion rate.

4) Effectiveness under Personalized Privacy Protection: in this section, we aim to evaluate the effectiveness of personalized privacy configuration within the FL_MADDPG algorithm. The personalized privacy configuration parameter, denoted as $\xi_{n,t}$, calibrates the perturbation level of the uploaded model update. Notably, a smaller value of ξ corresponds to a higher degree of privacy protection, which typically leads to a more significant impact on the model's performance and the accuracy derived from local data. To comprehensively assess the influence of different privacy configurations on system performance, we consider three distinct privacy protection scenarios. In our experiments, the global privacy budget threshold is set to $\varepsilon_{max} = 5$. The changes in global accuracy for the LISA, MNIST and CIFAR-10 datasets under various privacy scenarios are depicted in Fig. 7-Fig. 9. These scenarios reflect different task priorities and privacy requirements:

***SEC 1*:** in the scenario, participant privacy configurations are set to $\xi = 1.0$ and $\xi = 3.0$, inducing privacy budgets below the global privacy budget threshold $\varepsilon_{\max}$. Under the priority-conditioned calibration rule, a smaller privacy configuration yields a smaller privacy budget and stronger Laplace perturbation on the uploaded model updates. Consequently, the central server requires no additional aggregation-level perturbation. As shown in Fig. 7(a), Fig. 8(a), and Fig. 9(a), the stronger privacy settings yield lower global accuracy than the zero-noise baseline across all three datasets. Specifically, the $\xi = 1.0$ setting introduces the strongest perturbation and causes the largest accuracy degradation, whereas the $\xi = 3.0$ setting provides a milder privacy-utility trade-off. The results confirm that priority-conditioned privacy calibration strengthens protection for high-priority tasks, while the corresponding perturbation reduces the utility retained in the aggregated model updates.

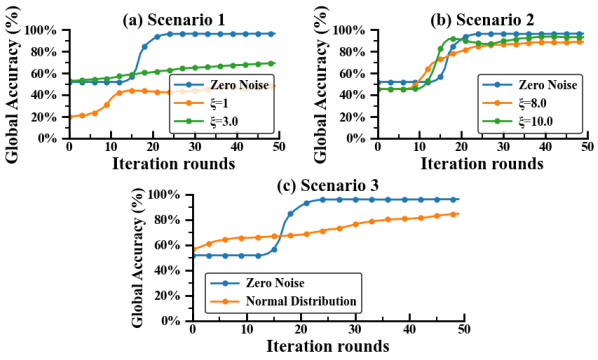


Fig. 7: Global accuracy of the LISA dataset model in multiple scenarios.

***SEC 2*:** in SEC 2, participant privacy configurations are set to $\xi = 8.0$ and $\xi = 10.0$, inducing privacy budgets above the global privacy budget threshold $\varepsilon_{\max}$. Under the priority-conditioned calibration rule, larger privacy configurations correspond to larger privacy budgets and weaker Laplace perturbations on the uploaded model updates. Consequently, the uploaded updates preserve more training information than updates generated under the stronger-privacy settings in SEC 1. During aggregation, the central server determines the need for

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

additional aggregation-level perturbation based on the global privacy budget threshold. As shown in Fig. 7(b), Fig. 8(b), and Fig. 9(b), the $\xi = 8.0$ and $\xi = 10.0$ settings achieve higher global accuracy and faster convergence than the stronger-privacy configurations. Specifically, the $\xi = 10.0$ setting yields the weakest perturbation, producing an accuracy curve closest to the zero-noise baseline. The results demonstrate that larger privacy configurations preserve model utility, although the corresponding privacy protection remains weaker than the protection assigned to high-priority tasks.

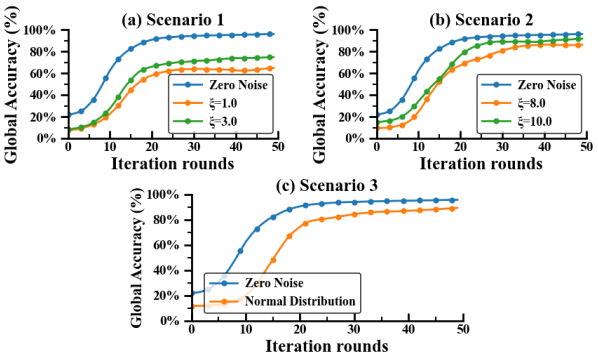


Fig. 8: Global accuracy of the MNIST dataset model in multiple scenarios.

***SEC 3*:** participant privacy configurations are sampled from a normal distribution to emulate a mixed-priority vehicular environment characterized by diverse task priorities and privacy requirements. Under the priority-conditioned calibration rule, the sampled configurations induce heterogeneous privacy budgets, yielding distinct perturbation strengths across the uploaded model updates. As shown in Fig. 7(c), Fig. 8(c), and Fig. 9(c), the mixed-privacy setting settles at an intermediate accuracy level between the strong-privacy configurations and the zero-noise baseline upon convergence. Compared with the fixed strong-privacy settings in SEC 1, the mixed-privacy setting avoids excessive perturbation across all updates to mitigate severe accuracy degradation, while maintaining stronger protection for high-priority tasks relative to the weak-privacy settings in SEC 2. Compared with the fixed privacy configurations in SEC 1 and SEC 2, SEC 3 shows the role of the proposed priority-conditioned privacy mapping. It assigns different perturbation levels to different task priorities, rather than enforcing a uniform perturbation strength for all uploaded updates. As a result, unnecessary accuracy degradation is reduced for updates associated with lower-priority tasks, while stronger protection is retained for high-priority task updates.

5) Privacy Attack: to evaluate the privacy protection capability of the proposed mechanism against attacks on uploaded model updates, we conduct a membership inference attack and an update reconstruction attack. For the membership inference attack, the area under the receiver operating characteristic curve (AUC) serves as the evaluation metric, where an AUC close to 0.5 indicates random guessing. As illustrated in Table IV, the attack performance varies across datasets under different privacy settings. On LISA, the AUC remains close to 0.5 under all settings, indicating limited membership leakage

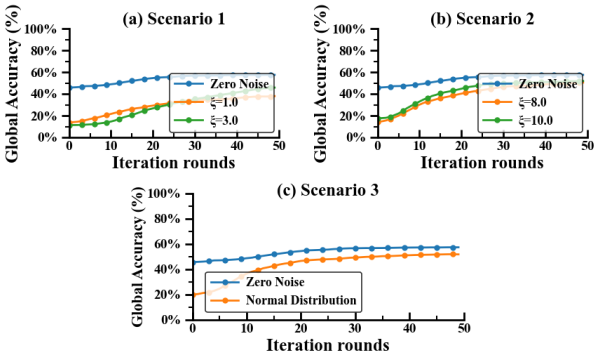


Fig. 9: Global accuracy of the CIFAR-10 dataset model in multiple scenarios.

from the uploaded updates. For MNIST and CIFAR-10, the zero-noise AUC values are 0.523 and 0.568, respectively. After perturbation is applied, the AUC values decrease to 0.505 and 0.522 at $\xi = 10$, and further approach the random-guessing level at $\xi = 1$. These results indicate that priority-conditioned perturbation limits the attacker's ability to infer whether a queried sample participates in local training.

TABLE IV: Mean AUC of membership inference attack.

Dataset	Zero Noise	$\xi = 10$	$\xi = 1$
MNIST	0.523	0.505	0.499
CIFAR-10	0.568	0.522	0.501
LISA	0.498	0.502	0.501

Additionally, we perform an update reconstruction attack as a complementary evaluation to assess whether the uploaded updates contain recoverable visual information. The attacker uses the global model before local training and the uploaded client update to optimize a dummy input, so that the gradient induced by the dummy input matches the target gradient estimated from the uploaded update. Fig. 10 presents the reconstruction results on LISA, MNIST, and CIFAR-10. For each dataset, (a) represents the original input, (b) displays the zero-noise reconstruction, while (c) and (d) illustrate the reconstructions at $\xi = 10$ and $\xi = 1$, respectively. Under the zero-noise setting, the uploaded updates retain recoverable visual structures, including the illuminated traffic-light region on LISA, the digit contour on MNIST, and the coarse color and shape information of the vehicle on CIFAR-10. After perturbation is applied, the reconstructed structures are substantially degraded. For LISA and MNIST, the reconstructions under both $\xi = 10$ and $\xi = 1$ collapse into smooth low-frequency regions, indicating that the perturbed updates contain limited stable sample-specific information. For CIFAR-10, the reconstruction under $\xi = 10$ still preserves partial color regions but disrupts the vehicle structure, while the reconstruction under $\xi = 1$ further loses recognizable semantic information.

Furthermore, the mean squared error (MSE) quantifies the pixel-level deviation between the reconstructed and original images, where a larger MSE signifies greater disparity. For the LISA, MNIST, and CIFAR-10 datasets, the zero-noise MSE values are 0.0007, 0.0035, and 0.0030, respectively. At $\xi = 10$, the MSE values increase to 0.1461, 0.2567, and 0.1648, further increasing to 0.1681, 0.2842, and 0.2121 at $\xi = 1$. The results

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

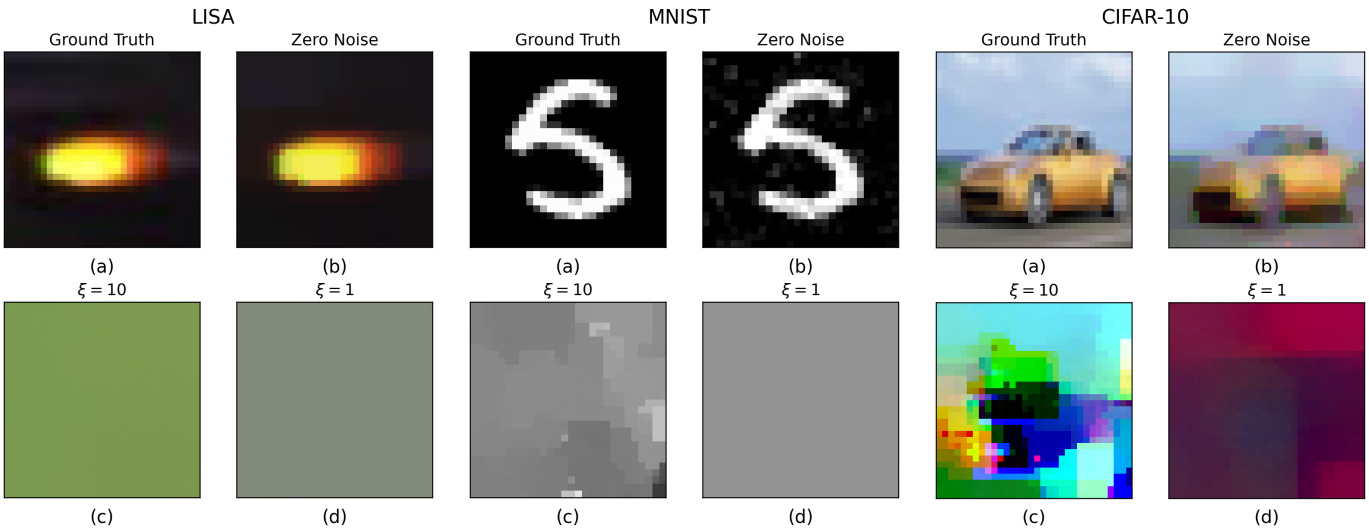


Fig. 10: Update reconstruction attack results on LISA, MNIST, and CIFAR-10. (a) denotes the original input; (b) denotes the reconstruction under Zero Noise; (c) and (d) denote the reconstructions under $\xi = 10$ and $\xi = 1$, respectively.

indicate that stronger priority-conditioned perturbation leads to larger reconstruction error and limits the sample-specific information recoverable from uploaded updates.

TABLE V: Comparison of component configurations under repeated runs

Method	Completion Rate (%)	Avg. Delay (s)	Avg. Energy (J)	Accuracy (%)
DRL-only	98.12 ± 0.05	0.5026 ± 0.0063	2.0815 ± 0.0319	88.74 ± 0.10
DP-only random	90.18 ± 0.00	1.6629 ± 0.0000	2.7441 ± 0.0000	69.94 ± 0.01
DP-only greedy	90.58 ± 0.00	1.3162 ± 0.0000	0.6667 ± 0.0000	79.28 ± 0.01
Joint	96.45 ± 0.21	0.7161 ± 0.0383	1.0566 ± 0.0665	82.53 ± 0.81

Note: DRL-only denotes FL_MADDPG without differential privacy, DP-only random and DP-only greedy remove the DRL-based scheduling policy, and Joint denotes the complete FL_MADDPG-based privacy-aware joint design. The results are reported as mean $\pm$ standard deviation.

6) Effect of Component Configuration: Table V presents the ablation results under different component configurations. The DRL-only setting achieves the highest accuracy of 88.74% and the lowest average delay of 0.5026 s, serving as an accuracy upper bound without privacy protection because no perturbation is applied to the uploaded model updates. However, the DRL-only baseline lacks privacy guarantees and incurs higher energy consumption than the complete Joint configuration. Without DRL-based scheduling, the two DP-only variants exhibit degraded task completion rates and prolonged delays. Specifically, the DP-only random setting yields a task completion rate of only 90.18%, along with the highest execution delay and energy consumption. The DP-only greedy setting reduces the average energy consumption to 0.6667 J and improves accuracy to 79.28%; nevertheless, the task completion rate remains at 90.58%, and the average delay is still significantly longer than that of the Joint configuration. In contrast, the Joint configuration achieves a 96.45% task completion rate, an average delay of 0.7161 s, and an average energy consumption of 1.0566 J, while maintaining an accuracy of 82.53% under privacy perturbation. Although the Joint configuration does not maximize every individual metric, the integrated design strikes a stable compromise among task completion, execution latency, energy consumption, and model utility. The results demonstrate that jointly integrating FL_MADDPG with priority-conditioned PDP achieves a more practical system balance than applying either DRL-based scheduling or privacy protection in isolation.

TABLE VI: Sensitivity analysis of reward weights under the Joint configuration.

Setting	(ν_d, ν_e, ν_c)	Completion Rate (%)	Avg. Delay (s)	Avg. Energy (J)
Default	$(0.30, 0.30, 0.40)$	96.45 ± 0.21	0.7161 ± 0.0383	1.0566 ± 0.0665
Delay-oriented	$(0.42, 0.18, 0.40)$	96.34 ± 0.33	0.5938 ± 0.0321	1.5365 ± 0.0646
Energy-oriented	$(0.18, 0.42, 0.40)$	98.69 ± 0.13	0.7777 ± 0.0575	0.7725 ± 0.1007
Constraint-oriented	$(0.18, 0.18, 0.64)$	95.43 ± 0.07	0.6485 ± 0.0455	1.6171 ± 0.1483

Table VI summarizes the impact of varying reward weight configurations on overall scheduling performance. The default setting achieves a completion rate of 96.45%, an average delay of 0.7161 s, and an average energy consumption of 1.0566 J, establishing a balanced performance baseline across the three metrics. Increasing the delay weight reduces the average delay from 0.7161 s to 0.5938 s, but raises the energy consumption to 1.5365 J and slightly decreases the completion rate to 96.34%, implying that a stronger delay preference tends to require higher energy consumption. In contrast, the setting with a larger energy weight reduces the average energy consumption from 1.0566 J to 0.7725 J while increasing the average delay to 0.7777 s, illustrating that aggressive energy conservation can prolong task execution. The setting emphasizing constraints yields the lowest completion rate of 95.43% alongside the highest energy consumption of 1.6171 J, which suggests that excessive feasibility penalties may induce conservative and less efficient scheduling decisions. Overall, the default reward weights strike a stable balance among task completion, execution delay, and energy consumption.

VI. Conclusion

Although FL_MADDPG performs well in terms of energy consumption and average task delay, especially with dynamic resource changes, there is still room for improvement in its

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

task completion rate in fixed, high-concurrency scenarios. In future research, we will refine the framework to improve task completion in congested scenarios while preserving its delay and energy advantages, and investigate lightweight and deployable variants to improve practical scalability.

VII. ACKNOWLEDGMENTS

This work is supported in part by the National Natural Science Foundation of China (No. 62302200, No. 62662042), the Natural Science Foundation of Jiangxi Province (No. 20252BAC220009, No. 20232BAB212014), the Key Laboratory of Multidimensional Intelligent Perception and Control of Jiangxi Province (No. 2024SSY03161).

REFERENCES

[1] P. S. Kumar, P. Saxena, and O. Alay, "Next-generation drl empowered actor-critic schedulers for multipath quic in 5g vehicular iot," Internet of Things, vol. 32, p. 101616, 2025.

[2] X. Fan, W. Gu, C. Long, C. Gu, and S. He, "Optimizing task offloading and resource allocation in vehicular edge computing based on heterogeneous cellular networks," IEEE Transactions on Vehicular Technology, vol. 73, no. 5, pp. 7175–7187, 2024.

[3] M. Tian, C. Li, Y. Hui, N. Cheng, and Z. Han, "On-demand services in 6g sdn/nfv-enabled space-air-ground-integrated vehicular networks," IEEE Wireless Communications, vol. 32, no. 3, pp. 156–163, 2025.

[4] H. Abumarshoud, L. S. Mohjazi, O. A. Dobre, M. D. Renzo, M. A. Imran, and H. Haas, "Lifi through reconfigurable intelligent surfaces: A new frontier for 6g?," IEEE Vehicular Technology Magazine, vol. 17, pp. 37–46, Mar 2022.

[5] S. Yu, X. Chen, Z. Zhou, X. Gong, and D. Wu, "When deep reinforcement learning meets federated learning: Intelligent multi-timescale resource management for mec in 5g ultradense network," IEEE Internet of Things Journal, vol. 8, pp. 5678–5689, Feb 2021.

[6] Y. Chen, J. Wu, J. Han, H. Zhao, and S. Deng, "A game-theoretic approach-based task offloading and resource pricing method for idle vehicle devices assisted vec," IEEE Internet of Things Journal, vol. 11, no. 12, pp. 21954–21969, 2024.

[7] K. P. Thakur and B. Palit, "A qos-aware uplink spectrum and power allocation with link adaptation for vehicular communications in 5g networks," IEEE Transactions on Network and Service Management, vol. 22, no. 1, pp. 592–604, 2025.

[8] X. Huang, Y. Chen, Z. Liu, S. Zhao, Z. Li, R. Chen, and Q. Guan, "Task offloading based on the fusion of model- and data-driven intelligence for vehicular edge computing networks," IEEE Transactions on Intelligent Transportation Systems, vol. 26, no. 6, pp. 8525–8540, 2025.

[9] S. Shen, G. Shen, Z. Dai, K. Zhang, X. Kong, and J. Li, "Asynchronous federated deep-reinforcement-learning-based dependency task offloading for uav-assisted vehicular networks," IEEE Internet of Things Journal, vol. 11, no. 19, pp. 31561–31574, 2024.

[10] G. Ma, M. Hu, X. Wang, H. Li, and Z. Chen, "Joint partial offloading and resource allocation for vehicular federated learning tasks," IEEE Transactions on Intelligent Transportation Systems, vol. 25, pp. 3456–3467, Aug 2024.

[11] Y. Chen, Z. Wang, H. Cai, Z. Qin, and S. Deng, "Federated knowledge distillation using hierarchical reinforcement learning in resource-constrained iot edge-cloud computing environments," IEEE Transactions on Mobile Computing, 2025.

[12] T. Zhao, F. Li, and L. He, "Secure video offloading in mec-enabled iiot networks: A multicell federated deep reinforcement learning approach," IEEE Transactions on Industrial Informatics, vol. 20, no. 2, pp. 1618–1629, 2024.

[13] F. Zheng, Y. Sun, and B. Ni, "Fedaeb: Deep reinforcement learning based joint client selection and resource allocation strategy for heterogeneous federated learning," IEEE Transactions on Vehicular Technology, vol. 73, no. 6, 2024.

[14] W. Mao, X. Lu, Y. Jiang, and H. Zheng, "Joint client selection and bandwidth allocation of wireless federated learning by deep reinforcement learning," IEEE Transactions on Services Computing, vol. 17, no. 1, pp. 336–348, 2024.

[15] R. Albelaihi, A. Alasandagutti, L. Yu, J. Yao, and X. Sun, "Deep-reinforcement-learning-assisted client selection in nonorthogonal-multiple-access-based federated learning," IEEE Internet of Things Journal, vol. 10, no. 17, pp. 15515–15525, 2023.

[16] Y. Chen, X. Zeng, H. Cai, Q. Xu, and Z. Liu, "Decentralized qos-aware model inference using federated split learning for cloud-edge medical detection," IEEE Transactions on Parallel and Distributed Systems, 2025.

[17] Z. Xu, A. D. Maio, E. Samikwa, and T. Braun, "Cstar-fl: Stochastic client selection for tree all-reduce federated learning," IEEE Transactions on Mobile Computing, vol. 24, no. 4, pp. 3110–3129, 2025.

[18] F. Shi, W. Lin, C. Peng, C. Zhong, D. Wang, and M. Cheng, "Dynamic client selection for over-the-air federated learning network," IEEE Internet of Things Journal, vol. 12, no. 12, pp. 18508–18519, 2025.

[19] Y. Chen, X. Zeng, X. Luo, and Z. Liu, "Joint client–server selection and resource allocation based on split federated learning in edge-to-cloud computing environments," Computer Networks, vol. 270, p. 111502, 2025.

[20] J. Tang, X. Li, H. Li, P. Li, X. Wang, and V. C. M. Leung, "Joint class-balanced client selection and bandwidth allocation for cost-efficient federated learning in mobile edge computing networks," IEEE Transactions on Mobile Computing, vol. 24, no. 7, pp. 5681–5698, 2025.

[21] J. Tan, Z. Liu, K. Guo, and M. Zhao, "Long-term client selection for federated learning with non-iid data: A truthful auction approach," IEEE Internet of Things Journal, vol. 12, no. 5, pp. 4953–4970, 2025.

[22] F. Sabah, Y. Chen, Z. Yang, A. Raheem, M. Azam, N. Ahmad, and R. Sarwar, "Fairdpfl-scs: Fair dynamic personalized federated learning with strategic client selection for improved accuracy and fairness," Information Fusion, vol. 115, p. 102756, 2025.

[23] Q. Li, S. Lyu, and J. Wen, "Optimal client selection of federated learning based on compressed sensing," IEEE Transactions on Information Forensics and Security, vol. 20, pp. 1679–1694, 2025.

[24] Q. Pan, H. Cao, Y. Zhu, J. Liu, and B. Li, "Contextual client selection for efficient federated learning over edge devices," IEEE Transactions on Mobile Computing, vol. 23, no. 6, pp. 6538–6548, 2024.

[25] L. Dong, Y. Zhou, L. Liu, Y. Qi, and Y. Zhang, "Age of information based client selection for wireless federated learning with diversified learning capabilities," IEEE Transactions on Mobile Computing, vol. 23, no. 12, pp. 14934–14945, 2024.

[26] C. Dong, F. Wu, Y. Qu, R. Akerkar, T. Wu, F. Zhou, Q. Wu, and S. Guo, "Joint data retransmission and client selection optimization for error-tolerant federated learning in aav networks," IEEE Transactions on Cognitive Communications and Networking, vol. 11, no. 4, pp. 2275–2288, 2025.

[27] J. Ma, Y. Zhou, Q. Li, Q. Z. Sheng, L. Cui, and J. Liu, "The power of bias: Optimizing client selection in federated learning with heterogeneous differential privacy," IEEE Transactions on Dependable and Secure Computing, vol. 22, no. 5, pp. 5672–5687, 2025.

[28] L. Chen, X. Ding, M. Li, and H. Jin, "Differentially private federated learning with importance client sampling," IEEE Transactions on Consumer Electronics, vol. 70, no. 1, pp. 3635–3649, 2024.

[29] X. Lu, Z. Liu, L. Xiao, and H. Dai, "Reinforcement learning-based personalized differentially private federated learning," IEEE Transactions on Information Forensics and Security, vol. 20, pp. 465–477, 2025.

[30] Q. Chen, Z. Ni, J. Li, and W. Liang, "Utility maximization of multi-federated learning in edge computing with personalized privacy preservation," in ICC 2025 - IEEE International Conference on Communications, pp. 2334–2339, 2025.

[31] J. Chen, H. Jiang, and Q. Hu, "Utility-enhanced personalized privacy preservation in hierarchical federated learning," IEEE Transactions on Mobile Computing, vol. 24, no. 6, pp. 5264–5279, 2025.

[32] M. Tahir, T. Mawla, F. Awaysheh, S. Alawadi, M. Gupta, and M. Intizar Ali, "Securefedprom: A zero-trust federated learning approach with multi-criteria client selection," IEEE Journal on Selected Areas in Communications, vol. 43, no. 6, pp. 2025–2041, 2025.

[33] J. Liu, Z. Chang, C. Ye, S. Mumtaz, and T. Hämäläinen, "Game-theoretic power allocation and client selection for privacy-preserving federated learning in iomt," IEEE Transactions on Communications, vol. 73, no. 8, pp. 5864–5880, 2025.

[34] C. Dwork and A. Roth, The Algorithmic Foundations of Differential Privacy, vol. 9. Foundations and Trends in Theoretical Computer Science, 2014.

[35] G. Nguyen, J. Sainz-Pardo Diaz, A. Calatrava, L. Berberi, O. Lytvyn, V. Kozlov, V. Tran, G. Moltó, and A. Lopez Garcia, "Landscape of machine learning evolution: privacy-preserving federated learning frameworks and tools," Artificial Intelligence Review, vol. 58, no. 2, p. 51, 2024.

[36] S. Li, W. Sun, Y. Sun, and Y. Huo, "Energy-efficient task offloading using dynamic voltage scaling in mobile edge computing," IEEE Transactions on Network Science and Engineering, vol. 8, no. 1, pp. 588–598, 2021.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3732546

[37] H. Jiang, X. Dai, Z. Xiao, and A. K. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," IEEE Transactions on Mobile Computing, 2022.
[38] H. Li, K. Xiong, Y. Lu, W. Chen, P. Fan, and K. B. Letaief, "Collaborative task offloading and resource allocation in small-cell mec: A multi-agent ppo-based scheme," IEEE Transactions on Mobile Computing, vol. 24, pp. 2346–2359, Mar 2025.
[39] H. Zhao, G. Lu, Y. Liu, Z. Chang, L. Wang, and T. Hämäläinen, "Safe dqn-based aoi-minimal task offloading for uav-aided edge computing system," IEEE Internet of Things Journal, vol. 11, pp. 32012–32024, Oct 2024.
[40] C. Li, K. Jiang, G. He, F. Bing, and Y. Luo, "A computation offloading method for multi-uavs assisted mec based on improved federated ddpg algorithm," IEEE Transactions on Industrial Informatics, vol. 20, no. 12, pp. 14062–14071, 2024.
[41] J. Xue, L. Wang, Q. Yu, and P. Mao, "Multi-agent deep reinforcement learning-based partial offloading and resource allocation in vehicular edge computing networks," Computer Communications, vol. 234, p. 108081, 2025.



Yishan Chen received the Ph.D. degree from the College of Computer Science, Zhejiang University, in 2022. She is currently working as an Associate Professor with Jiangxi University of Science and Technology. Her papers have been published in some well-known conference proceedings and international journals such as IEEE TMC, IEEE TPDS, IEEE TCC, IEEE TCE, IEEE ICWS, IEEE IOTJ, ACM TOSN, MONET, JOS, etc. Her research interests include Service Computing, Edge Intelligence, IoT.



Decan Cheng received the B.E. degree from the School of Computer and Information of Science, Hubei Engineering University, China, in 2023. He is currently pursuing the M.S. degree with the School of Information Engineering, Jiangxi University of Science and Technology, Ganzhou, China. His research interests include Edge Computing and Machine Learning.



Mengfan Teng received the Ph.D. degree from the School of Remote Sensing and Information Engineering, Wuhan University, in 2024. He is currently working as an lecturer with Jiangxi University of Science and Technology. His papers have been published in some international journals such as EI, STE, JCP, IEEE TGRS, etc. His research interests include Remote sensing estimation, Machine Learning, Air pollution.



Runshan Xie (Member, IEEE) received the Ph.D. degree from the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi, China, in 2024. He is currently working as a Lecturer with Jiangxi University of Science and Technology. His papers have been published in some well-known international journals such as *IEEE TFS*, *IEEE TII*, *Information Fusion*, and *Knowledge-Based Systems*, etc. His research interests include fuzzy systems, pattern recognition, and ensemble learning.



Junxiao Han (Member, IEEE) received the Ph.D. degree from the College of Computer Science, Zhejiang University, Hangzhou, China, in 2021. She is currently working as a Lecturer with Hangzhou City University, Hangzhou. Her papers have been published in some well-known conference proceedings and international journals, such as ICSE, ICSME, and EMSE. Her research interests include empirical software engineering, AI for SE, data science, and software repository mining.



Guanjie Cheng received the PhD degree in computer science and technology from Zhejiang University, in 2023. He is an assistant professor with the School of Software Technology, Zhejiang University. His research interests include edge intelligence, data governance, data security, and Internet of Things. He has published more than 20 papers in international conferences and journals such as IEEE Transactions on Information Forensics and Security, IEEE Transactions on Services Computing, and WWW.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 31,2026 at 06:21:48 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.